

Problem Setup

Task: Classify a **test point (1.6, 5.2)** into **Class A** or **Class B** using KNN.

Training Data:

Point	Feature 1 (X ₁)	Feature 2 (X ₂)	Class
P1	1.5	5.0	A
P2	1.8	5.0	A
P3	1.7	5.5	B
P4	2.0	5.4	B
P5	1.4	5.3	A

Parameters:

Distance metric: **Euclidean**

$K = 3$ (odd value to avoid tie)

Step 1: Calculate Distances

Euclidean distance formula:

$$d = \sqrt{(X_{1,test} - X_{1,train})^2 + (X_{2,test} - X_{2,train})^2}$$

Test point: (1.6, 5.2)

Computations:

$$d_{P1} = \sqrt{(1.6 - 1.5)^2 + (5.2 - 5.0)^2} = \sqrt{0.01 + 0.04} = 0.223$$

$$d_{P2} = \sqrt{(1.6 - 1.8)^2 + (5.2 - 5.0)^2} = \sqrt{0.04 + 0.04} = 0.283$$

$$d_{P3} = \sqrt{(1.6 - 1.7)^2 + (5.2 - 5.5)^2} = \sqrt{0.01 + 0.09} = 0.316$$

$$d_{P4} = \sqrt{(1.6 - 2.0)^2 + (5.2 - 5.4)^2} = \sqrt{0.16 + 0.04} = 0.447$$

$$d_{P5} = \sqrt{(1.6 - 1.4)^2 + (5.2 - 5.3)^2} = \sqrt{0.04 + 0.01} = 0.224$$

Step 2: Rank Neighbors by Distance

Point	Distance	Class	Rank
P1	0.223	A	1
P5	0.224	A	2
P2	0.283	A	3
P3	0.316	B	4
P4	0.447	B	5

Step 3: Select K-Nearest Neighbors

For $K = 3$:

Neighbors: P1 (Class A), P5 (Class A), P2 (Class A)

Step 4: Majority Voting

Class A votes: **3**

Class B votes: **0**

Prediction: Class A (unanimous)

Sensitivity Analysis: Impact of K

Case $K = 5$ (all neighbors):

Classes: A (P1, P5, P2), B (P3, P4)

Votes: A = **3**, B = **2** → **Class A** (still wins)

Case $K = 1$ (nearest neighbor):

Only P1 → **Class A**

Conclusion: Varying K preserves Class A as the dominant label.

Advanced Variation: Weighted KNN

If using **distance-based weighting** (closer neighbors = higher weight):

Weights: $w_i = \frac{1}{d_i}$

$$w_{P1} = \frac{1}{0.223} \approx 4.48$$

$$w_{P5} = \frac{1}{0.224} \approx 4.46$$

$$w_{p2} = \frac{1}{0.283} \approx 3.53$$

Weighted votes:

Class A: $4.48 + 4.46 + 3.53 = 12.47$

Class B: 0

Prediction: Class A (reinforced)

Key Insights from Evidence

1. **Distance Matters:** Euclidean is default, but Manhattan ($\sum |x_i - y_i|$) is an alternative.
2. **K Selection:** Odd K avoids ties; small K increases noise sensitivity, large K dilutes locality.
3. **Feature Scaling:** Unscaled features distort distance calculations (e.g., if X_1 range = $[0, 100]$ and X_2 range = $[0, 1]$).
4. **Weighted Schemes:** Prioritize closer neighbors to handle imbalanced data.

This example aligns with KNN's core mechanics: distance calculation, neighbor selection, and voting. For code implementations, see Scikit-learn's `KNeighborsClassifier` or manual Python logic.